

AutoRCA

Automated Log Monitoring & Error Analysis System

TECHNICAL DOCUMENTATION

Organization	Kona Software Lab LTD
Document	Technical Documentation
Version	1.0
Date	17 June 2026
Classification	Confidential

1. Document Purpose

This document describes, in full technical detail, what the AutoRCA system does, how each part of it works, and which technologies it is built on. It is intended for engineers who will operate, maintain, or extend the system.

2. Executive Overview

AutoRCA is a continuously-running service that watches a folder for incoming application **error log files**, automatically analyses them using an AI model, and produces clear **Root-Cause Analysis (RCA) reports**. The results are presented both as Markdown report files and through a polished web portal.

For every log file it processes, the system answers six questions:

- 1. **What happened?**
- 2. **Why did it happen?**
- 3. **What is the impact?**
- 4. **What is the root cause?**
- 5. **How do we fix it?** (concrete, ordered steps)
- 6. **How confident is this analysis?**

In addition, it classifies each problem by **level** (Application, Server, Database, or Network), extracts the **HTTP request/response** that failed, and — when a single log contains many failing endpoints — detects **every affected endpoint** and groups them **by error reason**.

The system is designed around three principles: **run forever and recover from restarts**, **never analyse the same file twice**, and **be understandable at a glance** by both engineers and non-engineers.

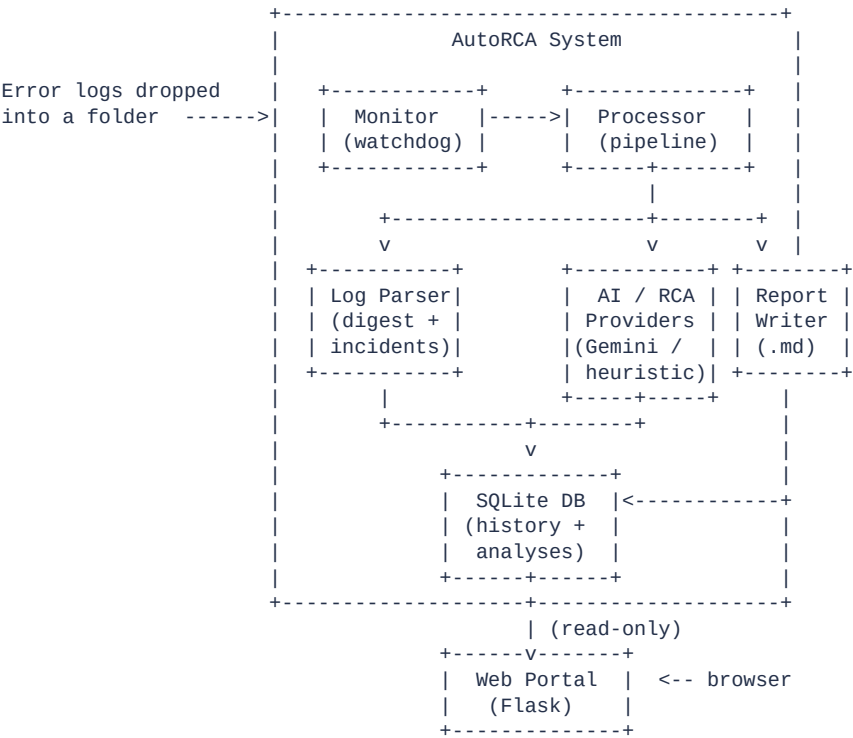
3. What the System Does (Functional Capabilities)

#	Capability	Description
1	Continuous folder monitoring	Watches a configured directory (recursively) for new and changed log files, reacting instantly to filesystem events and re-scanning every 30 seconds as a safety net.
2	Supported formats	Processes .log , .txt , .out , and .trace files.
3	Duplicate prevention	Each file is identified by a SHA-256 hash of its contents; an already-analysed file is never analysed again.
4	Restart recovery	All processing history is stored in a local database, so after a crash or restart the service resumes exactly where it left off.
5	AI root-cause analysis	Sends a distilled, token-efficient summary of the log to Google Gemini and receives a structured analysis.
6	Offline fallback engine	A built-in rule-based engine analyses logs with no internet/API key, and automatically takes over if the AI call fails.
7	Level classification	Pins each issue to one of four levels: Application / Server / Database / Network .

#	Capability	Description
8	Plain-English explanation	Provides a one or two sentence, non-technical description of the problem next to the technical detail.
9	Request / Response capture	Extracts the actual HTTP request and response bodies from the log (trimming large encrypted payloads).
10	Multi-endpoint detection	Detects every endpoint that failed in a log, with occurrence counts.
11	Reason grouping	Groups affected endpoints by error reason — the same reason is explained once; different reasons are explained separately.
12	Report generation	Writes a timestamped Markdown report per log file.
13	Web portal	A dashboard, searchable report list, and rich per-report view, with a dark/light theme toggle.
14	File lifecycle management	After analysis, the source log is moved into a processed/ sub-folder.

4. System Architecture

4.1 High-Level Components



The **Monitor** and **Web Portal** are two independent processes that share one SQLite database. The Monitor writes; the Portal reads. They can run at the same time.

4.2 Project Layout

```
AutoRCA/
+- main.py           # Monitor entry point (CLI)
+- webapp.py         # Web portal entry point
+- config.yaml       # All runtime settings
+- .env              # Secret: GEMINI_API_KEY (not committed)
+- requirements.txt  # Python dependencies
+- Start Monitor.bat # Double-click launcher for the monitor
+- Start Portal.bat  # Double-click launcher for the portal
+- Error Log File/   # Watched input folder
```

```

| +- processed/          # Logs are moved here after analysis
+- reports/              # Generated Markdown RCA reports
+- autorca/              # Application package
  +- config.py           # Loads config.yaml + .env into a typed Config
  +- database.py         # SQLite history & structured analysis storage
  +- log_parser.py       # Raw log -> ErrorDigest + Incident list
  +- processor.py        # Single-file processing pipeline
  +- service.py          # watchdog monitor + periodic rescan loop
  +- reporter.py         # AnalysisResult + digest -> Markdown report
  +- providers/         # Pluggable analysis engines
    +- base.py           # Provider interface + AnalysisResult model
    +- gemini_provider.py # Google Gemini (REST) provider
    +- heuristic_provider.py # Offline rule-based engine / fallback
  +- web/                # Flask web portal
    +- __init__.py       # App factory, routes, grouping helpers
    +- templates/        # base / dashboard / reports / report HTML
    +- static/style.css  # Dark + light theme styling

```

5. How It Works (Processing Pipeline)

The lifecycle of a single log file is as follows.

Step 1 — Detection

The **Monitor** (`autorca/service.py`) uses the `watchdog` library to receive operating-system filesystem events (file created / modified / moved). It also performs a full re-scan of the folder every `poll_interval_seconds` (30s) to catch anything the event stream missed (for example on network shares).

Step 2 — Stability check

A file that has just appeared may still be in the middle of being written. The Monitor waits until the file's size has stopped changing for `stability_seconds` (2s) before it is considered complete. This prevents reading half-written files.

Step 3 — Identity & de-duplication

The **Processor** (`autorca/processor.py`) computes a **SHA-256 hash** of the file's bytes. If the database already contains that hash with a `processed` status, the file is skipped and simply moved aside. Because the key is the content hash, an edited file (new content) is treated as new work, while an identical re-drop is ignored.

Step 4 — Parsing into a digest

The **Log Parser** (`autorca/log_parser.py`) reduces a potentially huge, noisy log into a compact **ErrorDigest** containing only what matters for analysis:

- exception class names and the **Caused by**: chain (deepest cause = root cause);
- severity counts (ERROR / WARN / FATAL ...);
- the affected application component (inferred from package names, ignoring

framework noise such as Spring / Netflix / Undertow);

- correlation / trace IDs;
- HTTP request/response lines and **bodies**;
- a trimmed, information-dense **excerpt** capped at `max_excerpt_chars` (16,000)

so the AI request stays small and within the free tier.

It also runs **incident extraction** (`parse_incidents`): it scans for every error HTTP response (4xx/5xx), pairs each with the most recent request to the same path, extracts a **reason** from the response body, and merges identical (`endpoint`, `reason`) pairs into a single incident with an **occurrence count**.

Step 5 — Analysis

The Processor passes the digest to the configured **provider**:

- **Gemini provider** (`gemini_provider.py`): builds a structured prompt and

calls the Gemini REST API, requesting a strict JSON object. The prompt includes the excerpt, the exception chain, the request/response bodies, the list of affected endpoints, and the list of distinct reasons to explain.

- **Heuristic provider** (`heuristic_provider.py`): matches the log against a

library of known error signatures (auth/JWT, NullPointerException, database/connectivity, timeout, out-of-memory, configuration). Used when the provider is set to `heuristic`, or automatically when the Gemini call fails (missing key, network error, quota, malformed response).

Both return the same **AnalysisResult** object (see §7), so the rest of the system is provider-agnostic.

Step 6 — Report generation

The **Reporter** (`autorca/reporter.py`) renders the **AnalysisResult** and digest into a Markdown report (`reports/RCA_<file>_<timestamp>.md`), including the problem summary, level visual, affected-endpoints table, per-reason sections, classification, resolution steps, sequence of events, and evidence.

Step 7 — Persistence

The result is written to the **SQLite database** — both the individual fields (for fast querying/filtering) and the complete analysis as JSON (for the portal to render the rich view).

Step 8 — File hand-off

The original log file is moved into `Error Log File/processed/`. If a file fails to process for any reason, the failure is recorded and the file is still moved aside, so the service never loops forever on a single bad file.

6. AI / Root-Cause Engine

6.1 Model

The default model is `gemini-2.5-flash` (Google Gemini), accessed over the public Generative Language **REST API**. The REST endpoint is used directly (via the `requests` library) rather than a client SDK, which removes any dependency on a fast-moving SDK and keeps the integration stable.

6.2 Why these settings

- `temperature = 0.2` — favour precise, deterministic analysis over creativity.
- `responseMimeType = application/json` — force a machine-parseable response.
- `thinkingConfig.thinkingBudget = 0` — disable the model's "thinking" tokens.

Gemini 2.5 models reason internally by default, which consumes the output budget and can truncate the JSON; structured extraction does not need it.

- `maxOutputTokens = 8192` — ample room for multi-reason analyses.

6.3 Resilience

Every failure mode (no API key, network error, HTTP 429 quota, HTTP 503, or unparseable JSON) is caught. If `fallback_to_heuristic` is enabled (default), the offline engine produces the analysis instead, so a file is never dropped. The report records which engine produced it.

6.4 Token efficiency

Only the distilled digest — not the raw log — is sent to the model. Long encrypted/base64 payloads inside request bodies are trimmed (keys preserved, long values shortened). Stack traces are de-duplicated and

capped. This keeps each request comfortably inside the Gemini free tier.

7. Data Model

7.1 AnalysisResult (the analysis contract)

Produced by every provider; the single source of truth for a report.

Field	Meaning
<code>summary</code>	One-line plain headline of the problem.
<code>simple_explanation</code>	1–2 sentence non-technical explanation.
<code>what_happened, why_it_happened, impact, root_cause</code>	The core narrative.
<code>resolution_steps[]</code>	Ordered, concrete fixes.
<code>confidence, confidence_reason</code>	High / Medium / Low and why.
<code>level, level_reason</code>	Application / Server / Database / Network.
<code>error_type, exception_class, affected_component, failure_point, probable_trigger, category</code>	Technical classification.
<code>sequence_of_events[]</code>	Reconstructed chronology.
<code>cascading_failures[]</code>	Secondary failures triggered.
<code>reason_groups[]</code>	One explanation per distinct error reason (multi-endpoint logs).
<code>engine, model</code>	Provenance of the analysis.

7.2 Incident (one affected endpoint)

`method, path, status, reason, request_body, response_body, count.`

7.3 Database schema (`processed_files` table)

Key columns: `content_hash` (unique), `file_name`, `status` (`processed/failed`), `report_path`, `processed_at`, plus the denormalised analysis fields (`summary`, `level`, `category`, `confidence`, `endpoint`, `response_status`, ...) and two JSON blobs — `incidents_json` and `analysis_json`. New columns are added to existing databases automatically via a lightweight migration step at startup.

8. Multi-Endpoint Detection & Reason Grouping

When a single log file contains many failing endpoints (common in gateway and payment-service logs), AutoRCA:

- Extracts incidents** — every 4xx/5xx response becomes a candidate incident, paired with its request body and a reason parsed from the response body (e.g. an error code such as `50_0004_757`, a `devMessage`, or the HTTP status).
- De-duplicates with counts** — identical (`endpoint`, `reason`) pairs are merged, so an endpoint that failed 235 times appears once with `×235`.

3. **Groups by reason** — incidents are bucketed by their reason. The same reason is explained **once** (covering all its endpoints); **different reasons** each receive their **own** explanation, level, and fix.
4. **Renders per group** — the portal shows each reason group as a card with the plain explanation, why, fix, reason code, and an expandable list of the exact endpoints (method, path, status, count) and their request/response bodies.
- The deterministic grouping (which endpoints belong to which reason) is computed in code for reliability; the AI provides the human explanation for each reason.

9. Web Portal

A read-only Flask application over the same database.

- **Dashboard** — live counters (total / successful / failed / high-confidence), breakdowns **By Level** and **By Confidence**, and a recent-analyses feed. Auto-refreshes so new results appear automatically.
- **All Reports** — a searchable list (by file name, error type, category, or summary).
- **Report detail** — the at-a-glance view: the problem and plain-English explanation, the four-level visual, affected endpoints grouped by reason with request/response, the what/why/impact cards, fix steps, an event timeline, cascading failures, and a Download-Markdown button.
- **Dark / light theme** — a top-bar toggle; the choice is stored in the browser's `localStorage` and applied before first paint to avoid flicker.
- **Robust startup** — `webapp.py` automatically selects the next free port if the default (5000) is busy and opens the browser on launch.

10. Technology Stack

Layer	Technology	Why
Language	Python 3.10+	Strong standard library, easy AI/SDK integration, fast to build and operate.
File monitoring	watchdog	Cross-platform OS-level filesystem event notifications.
Configuration	PyYAML + python-dotenv	Human-readable settings (<code>config.yaml</code>) with secrets isolated in <code>.env</code> .
HTTP / AI calls	requests	Direct, dependency-light access to the Gemini REST API.
AI model	Google Gemini <code>gemini-2.5-flash</code> (free tier)	Strong reasoning over stack traces; generous free quota.
Offline analysis	Custom rule engine	Signature-based RCA with no internet/API key; also the AI fallback.

Layer	Technology	Why
Storage	SQLite (Python <code>sqlite3</code>)	Zero-setup, file-based, transactional; perfect for processing history and restart recovery.
Web framework	Flask + Jinja2	Lightweight server-rendered portal.
Front-end	HTML + CSS (CSS custom properties for theming)	No build step; dark/light theming via CSS variables.
Packaging / run	Windows .bat launchers	One-click start for non-technical users.

External dependencies (all free / open-source): `watchdog`, `PyYAML`, `python-dotenv`, `requests`, `Flask`.

11. Configuration Reference (`config.yaml`)

Setting	Purpose	Default
<code>monitoring.watch_dir</code>	Folder to watch	<code>Error Log File</code>
<code>monitoring.recursive</code>	Watch sub-folders	<code>true</code>
<code>monitoring.extensions</code>	File types analysed	<code>.log .txt .out .trace</code>
<code>monitoring.stability_seconds</code>	Wait for a file to finish writing	<code>2</code>
<code>monitoring.poll_interval_seconds</code>	How often the folder is re-checked	<code>30</code>
<code>processing.processed_subdir</code>	Where processed logs are moved	<code>processed</code>
<code>processing.reports_dir</code>	Where reports are written	<code>reports</code>
<code>processing.db_path</code>	History database	<code>autorca.db</code>
<code>processing.max_excerpt_chars</code>	Max log size sent to the AI	<code>16000</code>
<code>ai.provider</code>	<code>gemini</code> or <code>heuristic</code>	<code>gemini</code>
<code>ai.model</code>	Gemini model id	<code>gemini-2.5-flash</code>
<code>ai.timeout_seconds</code>	API request timeout	<code>60</code>
<code>ai.fallback_to_heuristic</code>	Use the offline engine if AI fails	<code>true</code>
<code>logging.level / logging.file</code>	Log verbosity / file	<code>INFO / autorca_service.log</code>

The secret API key lives in `.env` as `GEMINI_API_KEY`, never in `config.yaml`.

12. Command-Line Interface

Monitor (`main.py`)

Command	Action
<code>python main.py</code>	Start continuous monitoring (runs until stopped).
<code>python main.py --once</code>	Process current files once, then exit.

Command	Action
<code>python main.py --file PATH</code>	Analyse a single file and exit.
<code>python main.py --stats</code>	Print processing-history statistics.
<code>python main.py --config PATH</code>	Use an alternate config file.

Portal (`webapp.py`)

Command	Action
<code>python webapp.py</code>	Start the portal (auto-opens the browser).
<code>python webapp.py --port 8080</code>	Use a specific port.
<code>python webapp.py --host 0.0.0.0</code>	Expose on the local network.
<code>python webapp.py --no-browser</code>	Do not auto-open the browser.

13. Extensibility

- **Add another AI backend** (e.g. Groq, Ollama, OpenAI-compatible): create a

class in `autorca/providers/` that subclasses `AnalysisProvider`, returns an `AnalysisResult`, and register it in `autorca/providers/__init__.py`. No other code needs to change.

- **Support more log formats / signatures**: extend the regexes in

`log_parser.py` or add signatures to `heuristic_provider.py`.

- **Change the model**: edit `ai.model` in `config.yaml`.

14. Operational Notes

- The Monitor and Portal are separate processes; run both to monitor *and* view.
- State lives in SQLite, so restarting either process is always safe.
- The free Gemini tier has per-minute rate limits; if many files are dropped at

once, some may fall back to the offline engine for that run — nothing is lost.

- On Windows, the system runs as a console application; it can be wrapped as a

Windows Service (e.g. with NSSM) or scheduled at startup via Task Scheduler.

AutoRCA — Automated Log Monitoring & Error Analysis · Kona Software Lab LTD